

Лабораторная работа 14. Структуры данных очереди.

1. Создать интерфейс для очереди из целых чисел со следующими методами:

```
T Dequeue();           // извлечь элемент из головы очереди
void Enqueue(T value); // поместить элемент в очередь
T Peek();              // показать элемент в голове, не удаляя его

bool IsEmpty();        // проверить, что очередь пуста
bool IsFull();         // проверка на заполнение
```

Создать собственную реализацию интерфейса «очередь» из п. 0. на основе массива.

Протестировать на следующих примерах:

а) В тестирующей программе должны обрабатываться строки, содержащие символы с ASCII от 32 до 127

Пример:

```
EAS * Yes * QUE * * * stop * * * IO * to * N *
```

В них отдельные символы интерпретируются как команды:

- любая заглавная латинская буква – поместить эту букву в очередь
- звездочка – извлечь из очереди и вывести на экран очередной элемент
- другие символы игнорируются.

В программе ввести с консоли строку, вывести результаты ее обработки таким командным интерпретатором.

б) В тестирующей программе, поместить в очередь все введенные строки (вводить из текстового файла). Используя только операции *enqueue* и *dequeue* выполнить циклический сдвиг элементов в очереди так, чтобы в ее начале (в голове, front) был расположен наибольший элемент – самая длинная строка.

2*. Язык Quack —язык, который фигурирует в одной из задач с [Internet Problem Solving Contest](#).

Требуется написать интерпретатор языка Quack.

Виртуальная машина, на которой исполняется программа на языке Quack, имеет внутри себя очередь, содержащую целые числа по модулю 65536 (то есть, числа от 0 до 65535, соответствующие беззнаковому 16-битному целому типу). Слово *get* в описании операций означает извлечение из очереди, *put* — добавление в очередь. Кроме того, у виртуальной машины есть 26 регистров, которые обозначаются буквами от 'a' до 'z'. Изначально все регистры хранят нулевое значение. В языке Quack существуют следующие команды (далее под и подразумеваются некие абстрактные временные переменные):

+	Сложение: get , get , put () mod 65536
-	Вычитание: get , get , put () mod 65536
*	Умножение: get , get , put () mod 65536
/	Целочисленное деление: get , get , put div (будем считать, что div 0 = 0)
%	Взятие по модулю: get , get , put mod (будем считать, что mod 0 = 0)
>[register]	Положить в регистр: get , установить значение [register] в
<[register]	Взять из регистра: put значение [register]
P	Напечатать: get , вывести в стандартный поток вывода и перевести строку
P[register]	Вывести значение регистра [register] в стандартный поток вывода и перевести строку
C	Вывести как символ: get , вывести символ с ASCII-кодом mod 256 в стандартный поток вывода
C[register]	Вывести регистр как символ: вывести символ с ASCII-кодом mod 256 (где — значение регистра [register]) в стандартный поток вывода
:[label]	Метка: эта строка программы имеет метку [label]
J[label]	Переход на строку с меткой [label]
Z[register][label]	Переход если 0: если значение регистра [register] равно нулю, выполнение программы продолжается с метки [label]
E[register1][register2][label]	Переход если равны: если значения регистров [register1] и [register2] равны, исполнение программы продолжается с метки [label]
G[register1][register2][label]	Переход если больше: если значение регистра [register1] больше, чем значение регистра [register2], исполнение программы продолжается с метки [label]

Q	Завершить работу программы. Работа также завершается, если выполнение доходит до конца программы
[number]	Просто число во входном файле — put это число

Формат входного файла

Входной файл содержит синтаксически корректную программу на языке Quack. Известно, что программа завершает работу не более чем за шагов. Программа содержит не менее одной и не более инструкций. **Метки имеют длину от 1 до 10 и состоят из цифр и латинских букв.**

Формат выходного файла

Выведите содержимое стандартного потока вывода виртуальной машины в выходной файл.

Примеры

input.txt	output
100 0 :start >a Zaend <a <a 1 + - >b <b Jstart :end P	5050